Snowflake — Professional Training (Detailed Study Notes)

Facilitator: Ramkumar J Dhamodharan
Format: 3-day instructor-led training with lectures, demonstrations, and hands-on labs
Case Study: ATLAS — a market-data platform for Fidelity Investments' Equity & Options trading desk

DAY 1 — Foundations & Data Ingestion

Day 1 Agenda

- Snowflake Essentials & Architecture
- Virtual Warehouses — Cost & Performance
- Databases, Schemas, and Data Modeling
- Data Ingestion Techniques: Batch, Continuous, Streaming, and Semi-Structured
- Demonstrations & Practices

1. Snowflake Architecture

The problem with traditional warehouses

In traditional on-premises data warehouses (Teradata, Oracle Exadata, older SQL Server appliances), **storage and compute are physically tied together**. If you needed more processing power, you had to buy more storage too — and vice versa. Scaling meant buying a bigger box, and every team competed for the same fixed pool of resources. One analyst's heavy query could slow down everyone else's reports.

Snowflake's answer: three decoupled layers

Snowflake physically separates the system into **three independent layers**, each of which scales on its own:

1. Storage Layer

- All data lives in **cloud object storage** (Amazon S3, Azure Blob, or Google Cloud Storage), managed entirely by Snowflake.
- Data is stored as **compressed, immutable files**. "Immutable" means files are never edited in place — when data changes, Snowflake writes new files and updates metadata pointers. This immutability is the foundation that makes features like Time Travel and Zero-Copy Cloning possible later.
- You pay only for the compressed bytes you actually store.

2. Compute Layer (Virtual Warehouses)

- Compute is delivered through **Virtual Warehouses** — these are simply clusters of compute machines that you can spin up and shut down on demand.
- Multiple warehouses can read the same data simultaneously without contending with each other. The ETL team, the BI team, and the data-science team can each have their own warehouse, fully isolated.
- Warehouses do the actual query execution: scanning, filtering, joining, aggregating.

3. Cloud Services Layer — "The Brain"

- This is the coordination layer that runs everything behind the scenes:
  - **Query Optimizer** — decides the most efficient execution plan.
  - **Metadata Management** — tracks every file, every micro-partition, every table version.
  - **Security** — authentication, access control, encryption key management.
  - **Transaction Management** — guarantees ACID behavior across concurrent users.

The Library Analogy

- **Storage = The Library** — all the books (data) sit on shelves, organized and preserved.
- **Warehouses = Reading Rooms** — independent rooms where people actually work with the books. You can open more rooms when it's busy and close them when it's quiet. People in one room never disturb people in another.
- **Cloud Services = The Librarian** — knows exactly where every book is, who's allowed to read what, and fetches the right material efficiently.

2. Micro-Partitions — How Snowflake Stores and Finds Data

- Behind every table, Snowflake automatically breaks the data into **micro-partitions**: small, immutable chunks of roughly **50–500 MB of uncompressed data** each.

- For **every column in every micro-partition**, Snowflake records metadata — most importantly the **MIN and MAX values**, plus counts of distinct values and nulls.

Why this matters: Partition Pruning

When you run a filtered query, for example:

```
WHERE trade_ts >= '2024-06-04'
```

Snowflake first consults the metadata. Any micro-partition whose MAX value for `trade_ts` is *earlier* than `2024-06-04` **cannot possibly contain matching rows**, so Snowflake skips it entirely without reading a single byte of it. This is called **pruning**.

The phrase to remember

> **"PARTITIONS SCANNED vs. PARTITIONS TOTAL"**

You'll find this in the Query Profile. If a table has 10,000 micro-partitions and your query scanned only 12, pruning is working beautifully. If it scanned 9,800 out of 10,000, pruning has failed — and your query will be slow no matter how big your warehouse is. This ratio is the single most important diagnostic number for query performance on large tables.

3. Virtual Warehouses — The Cost & Performance Dial

T-shirt sizing

Warehouses come in **T-shirt sizes: XS, S, M, L, XL, and up**. Each step up **doubles** the compute resources — and doubles the **credits per hour** consumed.

How to reason about cost

Don't just think in "credits per hour" — think in **credits per query**:

- A bigger warehouse costs more per hour but may finish the query in half the time. If a query takes 10 minutes on a Small but 5 minutes on a Medium, the cost is the same — and you got the answer twice as fast.
- The two questions that determine your bill: **What size is the warehouse? And how long does it stay awake?**

Two independent knobs — Scale UP vs. Scale OUT

These solve two completely different problems:

Knob	What it does	When to use it
Scale Up (bigger size)	More power for a **single heavy query**	One complex query is slow, spilling to disk, processing huge data volumes
Scale Out (multi-cluster)	More **copies** of the same-size warehouse that spin up automatically under concurrency	Many users hitting the system at once; queries are *queuing*, not slow individually

The Truck Analogy — match the level to the symptom

- **One slow query → get a bigger truck.** (Scale up — the load is too heavy for one vehicle.)
- **A queue of waiting queries → get more trucks.** (Scale out — each load is fine, there are just too many of them.)

Buying a bigger truck doesn't help when the problem is fifty deliveries waiting in line; adding trucks doesn't help when one single load is too heavy.

Two critical settings

- **Auto-Suspend** — the warehouse automatically shuts down after N seconds/minutes of inactivity, so you stop paying.
- **Auto-Resume** — the warehouse automatically wakes up the instant a query arrives.

The lesson on suspension policy

> **Aggressive auto-suspend is for SPIKY workloads with gaps. For a steady trickle of queries, let the warehouse stay warm.**

Why? Every time a warehouse suspends, its **local cache is lost**. If queries arrive every couple of minutes, an aggressively suspending warehouse keeps cold-starting — re-reading data from remote storage each time — which is both slower and ultimately not cheaper. Suspend aggressively only when there are genuine long gaps between bursts of activity.

4. Databases, Schemas, and Modeling for Engineering

The object hierarchy

```
Account → Database → Schema → Objects
```

Objects include: **Tables, Views, Stages, Pipes, File Formats, Integrations**, and more.

Every object can be referenced by its **FQN (Fully Qualified Name)**:

```
DATABASE.SCHEMA.TABLE
```

Using FQNs in production code removes ambiguity — your script behaves the same regardless of the session's current database/schema context.

Layered (Medallion-style) Architecture

Production data platforms organize data into progressive layers of refinement:

Layer	Purpose
RAW	Data lands here **exactly as it arrives** — untouched, the immutable **source of truth**. If anything downstream goes wrong, you can always rebuild from RAW.
STAGING	Data is **cleaned, typed, deduplicated, and conformed**. This is the "we trust this now" layer — bad records are rejected or quarantined, data types are enforced, duplicates removed, codes standardized.
ANALYTICS	**Curated, business-ready tables and views** — typically dimensional models (star schemas) — that BI tools and analysts actually query.

Table Types — choosing the right durability/cost trade-off

Type	Time Travel	Fail-Safe	Cost	Use For
Permanent (default)	Yes	Yes (7 days)	Highest	Anything that matters — production data you can't recreate
Transient	Shorter	**No**	Cheaper	Re-loadable data

Type	Time Travel	Fail-Safe	Cost	Use For
Temporary	Session only	No	Minimal	Scratch work — **vanishes when your session ends**

Practical question from the course: *"Can you reload the landing tables from S3 buckets?"* — **Yes? Then make them transient.** There's no point paying Fail-Safe storage premiums to protect data you can re-copy from the bucket any time.

Row vs. Column Storage — why wide joins are cheap

- **OLTP systems (transactional)** use **row storage** — entire rows are stored together, optimal for "fetch/update this one record."
- **OLAP systems (analytical)**, including Snowflake, use **column storage** — each column's values are stored together, optimal for "scan three columns across a billion rows."

Because of columnar storage, **joining to wide dimension tables is cheap in Snowflake** — the engine only reads the specific columns your query touches, no matter how many columns the dimension table actually has. This removes the old pressure to keep dimensions narrow.

5. Getting Data In — Ingestion Mental Models

The three "doors" into Snowflake

Door	Tool	When	Data Unit
Batch / On-Demand	`COPY INTO`	"Load this pile of files **now**"	File(s)
Files, Continuously	**Snowpipe**	"Load each file **as it lands**"	File(s)
Rows, Continuously	**Kafka / Snowpipe Streaming**	"Load events in **near real-time**"	Row(s)

The Stage — the key concept for bulk loading

A **stage** is simply **a pointer to where files live**. It can be internal (Snowflake-managed) or external (your own S3/Azure/GCS bucket). The `COPY INTO` command reads files from a stage into a table.

Storage Integration — the Secure Handshake

A **Storage Integration** is a **trust relationship between your Snowflake account and an AWS IAM role**. It lets Snowflake read your bucket *without anyone ever embedding AWS access keys in SQL scripts* — a major security win.

Step-by-step setup process:

1. **In Snowflake:** create a **Storage Integration** pointing at your S3 bucket.
2. Run `DESCRIBE INTEGRATION` — Snowflake gives you two values: the **IAM User ARN** and an **External ID**.
3. **In AWS:** configure the **IAM role's trust policy** with those two values (so only *your* Snowflake account can assume the role).
4. Ensure the IAM role itself has **S3 read permissions** on the bucket.
5. **Back in Snowflake:** create the **stage** that uses the integration.

The three classic load failures

When a `COPY INTO` fails, it's almost always one of three things:

1. **The file format doesn't match the file** — e.g., the format says comma-delimited but the file is pipe-delimited, or the header-skip setting is wrong.
2. **A column count mismatch** — the file has more or fewer columns than the target table expects.
3. **A date or number that won't parse** — a value like `2024-13-45` or `N/A` in a numeric column.

Diagnose these by checking the error message and validating the file format against an actual sample of the file.

Load History — built-in idempotency

Snowflake **remembers the metadata of which files have already been loaded** into each table. If you run the same `COPY INTO` again, already-loaded files are silently skipped — protecting you from accidental duplicates. This memory has a **load history window** (a retention period), after which the same file could be loaded again, so keep that window in mind for long-lived pipelines.

6. Semi-Structured Data — JSON Without the Pain

VARIANT — one column to hold it all

- **VARIANT** is a special data type: **a single column that holds an entire JSON document** (or array, or any semi-structured value).

- You don't have to define a rigid schema before loading. Land the raw JSON first, figure out the structure later.

Querying inside JSON with path notation

```
SELECT
    raw:symbol::string,
    raw:price::float
FROM market_ticks_json;
```

- `raw:symbol` — navigates into the JSON using `:` **path notation**.
- `::string` / `::float` — **casts** the extracted value to a proper SQL type.

LATERAL FLATTEN — exploding arrays into rows

When a JSON document contains an **array** (e.g., one options-chain document containing 200 contracts), `LATERAL FLATTEN` **explodes the array into individual rows** — turning one nested document into a proper relational table that you can join and aggregate normally.

The philosophy: SCHEMA-ON-READ

> **Load now, define the shape later.**

Traditional warehousing is *schema-on-write* (define the table perfectly, then load). Snowflake's VARIANT enables *schema-on-read*: capture data immediately as it arrives, and impose structure at query time. This is invaluable when source systems change their JSON payloads frequently — your ingestion never breaks; only your parsing views need updating.

7. Snowpipe — Continuous File Ingestion

The core idea

> **Snowpipe is a COPY that fires automatically the moment a file appears.**

It uses the exact same load logic as `COPY INTO` — same file formats, same transformations — but the **trigger is the file's arrival**, not your hand on the keyboard.

How Snowpipe really works (the AWS event chain)

1. A **file lands in the S3 bucket**.

2. S3 fires an **event notification** to an **SQS queue**. (Snowflake gives you the Queue ARN to point your bucket's event notification at.)
3. **Snowpipe sees the SQS message** and runs the pipe's **predefined COPY command** for that one file.
4. Rows show up in your table **seconds to minutes later**.

Key characteristics

- **SERVERLESS** — there is **no warehouse for you to manage and no paying to keep compute warm**. Snowflake provisions compute per file behind the scenes and bills per usage.
- **Per-file cost component** — Snowpipe charges include a small overhead **per file**, so **the shape of your files matters a lot**: thousands of tiny 10 KB files are far more expensive per gigabyte than a steady flow of well-sized files. Aim for sensibly batched file sizes (Snowflake's general guidance is files in the ~100–250 MB compressed range where possible).

8. Kafka & Snowpipe Streaming — True Real-Time

When even "file lands → seconds later" isn't fast enough — e.g., a live trade-execution feed — you skip files entirely and stream **rows** directly.

How it works

1. A **Kafka topic** carries the events (e.g., market-tick / order / fill events).
2. The **Snowflake Sink Connector**, running inside your Kafka Connect cluster, **subscribes to the topic and writes straight into the Snowflake table**.
3. A connector configuration option — `ingestion.method = SNOWPIPE_STREAMING` **vs.** `SNOWPIPE` — controls the behavior:
   - **SNOWPIPE_STREAMING** → rows are written **directly** into the table with low latency (rows-direct).
   - **SNOWPIPE** (legacy mode) → the connector batches rows **into files** first, then loads them — the old, higher-latency behavior.

Authentication: Key-Pair

- The connector authenticates with **key-pair authentication** (a private/public key pair).
- The connector **signs in with the private key**, and Snowflake **verifies it against the public key you registered on the user**.
- No passwords are stored in connector configs — this is the standard, secure pattern for all service accounts.

9. The Capstone Case Study — ATLAS (Fidelity Investments)

Scenario: Fidelity's **E&O (Equity & Options) Trading Desk** is consolidating its entire market-data estate into Snowflake. The platform's codename is **ATLAS**.

The business requirement

Ingest market activity at **three different velocities** and serve it to **analysts, risk, and compliance from one place**:

1. **Historical** — years of equity ticks for **back-testing and TCA** (Transaction Cost Analysis).
2. **Batch** — nightly trade files dropped by venues (e.g., NYSE) into S3.
3. **Micro-Batch** — intraday tick files that land throughout the day and must be loaded **within seconds** (→ Snowpipe).
4. **Streaming** — a live execution feed (orders/fills) flowing through **Kafka**, landing in Snowflake with low latency (→ Snowpipe Streaming).

Along the way, the data is **messy, semi-structured JSON** (nested option chains) that must be modeled into clean **STAR schemas**.

The ATLAS architecture (end-to-end flow)

Sources → AWS Landing Zone → Snowflake (FIDELITY_ATLAS) → Consumers

- **Market Data Sources:**
  - NYSE / NASDAQ — nightly batch CSV
  - CBOE options — nested JSON
  - Venue intraday tick drops
  - Live execution feed
- **AWS Landing Zone:**
  - `S3 /batch` (Lab 4 — bulk COPY)
  - `S3 /json` (Lab 5 — semi-structured)
  - `S3 /autoingest` + **SQS queue** (Lab 6 — Snowpipe)
- **Local Docker (Lab 7):** Kafka broker + Kafka Connect
- **Snowflake — Database `FIDELITY_ATLAS`:**
  - **Compute (Labs 1 & 2):** multi-cluster warehouses, pruning-aware queries
  - **RAW layer:** `TICKS` (clustered, Lab 1), `RAW_TRADES` (Lab 4), `RAW_OPTIONS` (VARIANT, Lab 5), `RAW_TICKS_STREAM` (Lab 6), `RAW_EXECUTIONS` (Lab 7)

    - **ANALYTICS layer (Lab 3):** dimensions `DIM_SECURITY / DIM_VENUE / DIM_DATE` and facts `FACT_TRADES / FACT_QUOTES / FACT_EXECUTIONS`
- **Consumers:** Trading analysts; Risk & compliance.

The lab capabilities being exercised

- **Pruning experiments** — *Why are tick queries slow?*
- **Scaling & concurrency test** — *Market open overwhelms the desk.*
- **Star-schema modeling challenge** — *Analysts can't read raw feeds.*
- **Bulk load + S3 integration**
- **Semi-structured JSON handling**
- **Snowpipe auto-ingest (SQS)** — *Nightly is too slow.*
- **Kafka / Snowpipe Streaming** — *The execution feed must stream in real time.*

DAY 2 — Transformation, Orchestration & Performance

Day 2 Agenda

- Demonstrations of Day 1 (continued)
- Streams (CDC)
- Tasks & Scheduling
- Dynamic Tables
- Apache Airflow (Enterprise Orchestration with Snowflake)
- Advanced SQL
- Query Performance Tuning
- Clustering, Search Optimization, Materialized Views
- Caching & Cost Governance
- Demonstrations & Practices

10. Streams — Change Data Capture (CDC)

The problem

How do we process ONLY what's new? Reprocessing an entire billion-row table every night just to pick up yesterday's changes is wasteful. We need a way to see only the delta.

The concept: a Stream is a bookmark

- A **stream is a bookmark on a table**.
- It does **NOT copy the data**. It simply **remembers an offset** — a point in the table's change history — and when queried, shows you **everything that changed since you last looked**: INSERTs, UPDATEs, and DELETEs, each tagged with metadata columns (`METADATA$ACTION` , `METADATA$ISUPDATE` , etc.).

The "unread messages" analogy

> A stream is the **unread-messages badge on your inbox**. **Reading the messages — consuming the stream inside a DML statement** (an INSERT/MERGE that uses it) — **clears the badge** and advances the offset. **Just peeking with a plain SELECT does NOT** clear it.

This distinction is crucial: you can inspect a stream's contents harmlessly all day with SELECT; the offset only moves forward when the stream is used inside a successfully committed DML transaction.

The staleness trap

A stream can go **stale** if it remains **unconsumed past the table's retention window** (the Time-Travel retention period). Once stale, the stream can no longer reconstruct the changes and must be recreated. Lesson: consume your streams regularly, or extend the retention period on heavily changing tables.

11. Tasks — Native Scheduling Inside Snowflake

The need

Scheduled execution that **lives where the data lives**, with **auditable history** — without standing up an external scheduler VM.

The concept

- A **task is a scheduled SQL statement (or stored procedure call) inside Snowflake**.
- Tasks **chain into DAGs** (Directed Acyclic Graphs): a **root task** runs on a schedule, and **child tasks fire after their parents complete** — letting you build multi-step pipelines natively.

Two flavors of compute

Flavor	Who picks the compute
Warehouse Tasks	**You** choose the warehouse it runs on
Serverless Tasks	**Snowflake** sizes the compute per run automatically

The killer combination: Tasks + Streams

```
WHEN SYSTEM$STREAM_HAS_DATA('stream_name')
```

Adding this clause to a task makes it **conditional on real changes**: the task is scheduled to wake up, but it only actually runs (and only consumes credits) **if the stream has new data**. No changes → no run → no cost.

Auditability

Every execution is recorded in `TASK_HISTORY` — start time, end time, status, error messages. The result is a pipeline that is:

- **Scheduled**
- **Conditional on real changes**
- **Fully audited**
- **Requires no VM** to maintain

12. Dynamic Tables — The Declarative Alternative

The pain point

For complex pipelines, managing **Streams + Tasks + MERGE statements + the dependency graph** between them requires significant review, maintenance, and operations effort. Every table needs its stream, its task, its merge logic, and careful ordering.

The question

Can we just declare WHAT each table should contain and let Snowflake figure out HOW to keep it fresh?

The concept

> **A dynamic table is a SELECT statement plus a freshness promise.**

```
CREATE DYNAMIC TABLE positions
  TARGET_LAG = '5 minutes'
  WAREHOUSE  = 'DEMO_WH'
AS
SELECT ... ;
```

- `TARGET_LAG` is the freshness promise: "this table's contents should never be more than 5 minutes behind its sources."
- Snowflake **watches the source tables, computes incremental refreshes itself**, and when you build **dynamic tables on top of other dynamic tables**, it automatically assembles and manages the **dependency graph** — refreshing everything in the correct order.

The analogy

> **Streams + Tasks = a manual-gear car.** You control every shift — maximum control, maximum effort.
> **Dynamic Tables = an automatic-gear car.** You declare the destination (the SELECT) and the speed (the lag); the car handles the gears.

Real-world impact

For a **40-table analytics layer**, the Streams+Tasks approach means ~40 streams, ~40 tasks, ~40 MERGE statements, and a hand-managed dependency graph. Dynamic tables **collapse all that plumbing into ONE declaration per table**.

13. Apache Airflow — Enterprise Orchestration

The driver

The compliance team wants **ONE place to see every overnight job across the firm** — not one console per tool (one for Snowflake tasks, one for the Spark cluster, one for the file movers...).

The concept

- **Airflow** is an **open-source, license-free (Apache License), lightweight, general-purpose orchestrator**.
- **Pipelines are Python DAGs**; the steps are **Operators** — run SQL, call an API, run Python, wait on a sensor — with built-in **retries, alerting, backfills, and one UI for everything**.
- Snowflake integrates via dedicated operators/hooks, so Airflow can trigger and monitor Snowflake SQL alongside everything else in the enterprise.

The Rule of Thumb — when to use which orchestrator

Workload	Right tool
In-database, data-driven steps	**Native Tasks / Dynamic Tables** — keep it inside Snowflake
Cross-system workflows (touching S3, APIs, other databases, notifying humans)	**Airflow** — the general-purpose conductor

14. Advanced SQL Capabilities

Three important tools, one purpose each:

a) Window Functions — compute across rows without collapsing them

- A normal `GROUP BY` aggregation collapses rows. A **window function** computes aggregate-style results **OVER** a window of rows defined by `PARTITION BY` and `ORDER BY` — while **keeping every row** in the output.
- Example uses: running totals, moving averages, rank-within-group, comparing each trade to the prior trade.
- **`QUALIFY` for deduplication** — Snowflake's `QUALIFY` clause filters on window-function results directly. The classic dedup pattern:

```
SELECT *
FROM raw_trades
QUALIFY ROW_NUMBER() OVER (PARTITION BY trade_id ORDER BY load_ts DESC) = 1;
```

This keeps only the latest version of each `trade_id` — no subquery gymnastics required.

b) MERGE — upsert in one atomic statement

- **MERGE** performs **"update if it exists, insert if it doesn't" in one atomic statement** — comparing a source (often a stream) against a target table with `WHEN MATCHED THEN UPDATE` / `WHEN NOT MATCHED THEN INSERT` clauses.
- It is the workhorse of CDC pipelines: stream feeds the MERGE, MERGE keeps the target current.

c) Stored Procedures — Snowflake Scripting

- **Snowflake Scripting** lets you wire **multi-step logic — variables, control flow (IF/FOR/WHILE), and exception handling — into ONE callable unit** invoked with `CALL procedure_name(...)` .
- Use procedures for operational logic that's awkward as a single SQL statement: conditional loads, retry wrappers, administrative loops.

15. Query Performance Tuning — Diagnose Before You Spend

> **Fidelity needs engineers who diagnose before they spend.** Upsizing the warehouse is not the universal fix — sometimes it's exactly the wrong fix.

The diagnostic instrument: the QUERY PROFILE

The Query Profile (in Snowsight) visualizes how a query actually executed — every operator, row counts in and out, bytes scanned, spillage. Learn to read it, and **look for three patterns**:

Pattern 1 — Poor Pruning

- **Symptom:** *Partitions Scanned ≈ Partitions Total.* The query reads (almost) the whole table.
- **Cause:** the **filter column's values are not organized** across micro-partitions — every partition contains the full range of values, so none can be skipped.
- **Fix: CLUSTERING** the table on the filter column.
- **NOT a bigger warehouse** — a bigger warehouse just scans the same wasted partitions faster, at double the cost.

Pattern 2 — Spillage

- **Symptom:** the profile shows **"Bytes spilled to Local Storage"** or worse, **"Bytes spilled to Remote Storage."** The operation (a big sort, join, or aggregation) didn't fit in the warehouse's

memory and overflowed to disk — remote spillage is dramatically slower.
- **Fix:** this is the one case where a **BIGGER WAREHOUSE** genuinely helps (more memory) — **or restructure the query** to process less data at once (filter earlier, aggregate before joining, etc.).

Pattern 3 — Join Explosion

- **Symptom:** a **join node emits far more rows than it receives** — e.g., 1M rows in, 80M rows out.
- **Cause: duplicate join keys multiplying rows** — the join key isn't unique on the side you assumed it was, creating an accidental many-to-many.
- **Fix: the join logic itself** — deduplicate the dimension, fix the key, tighten the join condition.
- **NO HARDWARE HELPS.** This is a correctness bug; throwing compute at it just produces the wrong answer faster.

16. Clustering, Search Optimization & Materialized Views — The Decision Triangle

> **One table, three problems — there's NO single fix.** Each accelerator targets a different query shape, and each has its own ongoing cost.

1. Clustering Key → for RANGE SCANS

- **Reorganizes micro-partitions around your key** (e.g., `CLUSTER BY (trade_date)`), so range filters prune effectively.
- Snowflake runs **background reclustering**, which **costs credits** continuously as new data arrives.
- **Worth it for the DOMINANT filter** — the column that the majority of heavy queries filter on. Don't cluster for an occasional query pattern.

2. Search Optimization → for POINT LOOKUPS

- Builds a **persistent search structure for EQUALITY predicates**.
- Effect: finding `ACCT83498` in a multi-billion-row table goes from a **FULL SCAN to an INDEX-LIKE SEEK**.
- You **pay storage + maintenance** for the search structure.
- It is **POINTLESS for range scans** — it only accelerates equality (and similar point-lookup) predicates.

3. Materialized Views → for REPEATED AGGREGATIONS

- **Pre-compute the rollup once**; hundreds of dashboard hits then read the stored result. Snowflake keeps it **automatically refreshed** as the base table changes.
- The cost: you **pay maintenance on EVERY base-table change**.
- Therefore it's **WRONG for rarely-read aggregations over hot tables** (tables with billions of constantly changing rows) — you'd pay continuous refresh costs for a result almost nobody reads.

How to choose (summary)

Your query pattern	The right tool
Range scans on one dominant column	**Clustering key**
Needle-in-haystack equality lookups	**Search Optimization**
Same aggregation read over and over	**Materialized View**

17. Caching & Cost Governance

> Fidelity needs **attribution** (who spends what, how much, when) and **circuit breakers** (hard limits). Three layers:

Layer 1 — Caches

- **Result Cache:** if the **identical query** runs again (and the underlying data hasn't changed), Snowflake returns the previous result **in milliseconds at ZERO compute cost**. It's account-wide and lasts ~24 hours.
- **Warehouse Local Cache:** a running warehouse keeps recently read data on its local SSDs, making **warm warehouses faster** for subsequent related queries. (This is the cache you lose on auto-suspend — recall the Day 1 lesson.)
- **The gotcha:** putting `CURRENT_TIMESTAMP` (or any non-deterministic function) in a query **defeats the result cache entirely** — the query text resolves differently every run, so it never matches. Strip incidental non-determinism from dashboard queries.

Layer 2 — Resource Monitors: the circuit breaker

- A **credit quota per warehouse or per account**, with **thresholds** that can notify at (say) 75%, and **suspend the warehouse** at 100%.

- This is the hard stop that prevents a runaway job from burning a month's budget over a weekend.

Layer 3 — Attribution: who spent what

Built on the `ACCOUNT_USAGE` schema plus **query tagging**:

- **Query Tagging** — set a tag (team / job / project) on sessions or statements so every query carries an attribution label.
- `QUERY_HISTORY` — every query, its duration, its warehouse, its tag → **rank the most expensive queries**.
- `WAREHOUSE_METERING_HISTORY` — credit consumption per warehouse per hour → **splits spend by warehouse** (and thus by the team that owns it).

DAY 3 — Advanced Features, Security, Snowpark & AI

Day 3 Agenda

- Demonstrations & Practices — Day 2 topics; Caching & Cost Governance
- Time Travel & Zero-Copy Cloning
- Secure Data Sharing
- RBAC (Role-Based Access Control)
- Data Security: Masking, Row-Level Security & Tags
- Security & Audit
- Snowpark
- Migration & Engineering Disciplines
- Agentic AI on Snowflake (ML → Generative AI → Agentic AI)
- Demonstrations & Practices

18. Time Travel & Zero-Copy Cloning

Time Travel — querying the past

- **Every change to a table is retained for a window — 1 day by default, configurable up to 90 days on Enterprise edition**.
- You can **query the table as it existed in the past** directly, using the `AT` or `BEFORE` operators:

```
-- The table as it was 5 minutes ago
SELECT * FROM trades AT(OFFSET => -60*5);

-- The table as it was just BEFORE a specific (disastrous) statement ran
SELECT * FROM trades BEFORE(STATEMENT => '<query_id>');
```

- `UNDROP TABLE|SCHEMA|DATABASE` — **resurrects the whole object** instantly if someone drops it within the retention window.

The hero scenario: someone runs an **UPDATE without a WHERE clause** and mangles a production table. The fix is **one statement** — re-insert from the table *as it was before that query ID* — recovered **in seconds, not hours** of restore-from-backup.

Zero-Copy Cloning — instant environments

```
CREATE TABLE dev_trades CLONE prod_trades;
```

- Cloning **duplicates metadata pointers, NOT data** — it is **instant** and consumes **near-zero storage** at creation, regardless of table size.
- This works because micro-partitions are immutable: clone and original safely share the same underlying files.
- **The clone only pays for blocks that diverge after cloning** — i.e., new or modified micro-partitions unique to one side.
- **The payoff: full production-shaped dev/test environments stop being a storage negotiation.** Clone the entire production database for a test run, then drop it — cost is negligible.

19. Secure Data Sharing

The requirement

Grant **LIVE, read-only, revocable** access to another organization **without the data ever moving** — no nightly extract files, no SFTP, no stale copies.

The concept: a Share

- A **share grants another Snowflake account read access to your storage** — **NO COPY, NO MOVEMENT**.
- The **consumer queries your live data using their own compute** (their warehouses, their credits), and **you can revoke access instantly**.
- Because it's the same underlying storage, the consumer always sees current data — there is no sync lag and no divergent copy floating around.

Controlling exactly what they see

- **Wrap the exposure in SECURE VIEWS** so consumers see **exactly the columns/rows you choose** — and **nothing about the underlying definition** (secure views hide their SQL text and block optimizer-based inference of hidden data).

Partners without a Snowflake account

- Provision a **Reader Account** — a managed account that **they log into but you provision and whose compute you control** (and pay for). This extends sharing to partners with zero Snowflake footprint.

20. RBAC — Role-Based Access Control

The model

- **Privileges attach to ROLES → roles are granted to USERS → and roles can be granted to other ROLES**, forming a hierarchy.
- You never grant privileges directly to a human; you grant them to roles.

The discipline that scales: two layers of custom roles

Layer	Answers	Contains
Access Roles	**WHAT**	Bundles of privileges on objects (e.g., `SALES_DB_READ` = USAGE on db/schema + SELECT on tables)
Functional Roles	**WHO**	Personas (**Advisor, Analyst, Compliance**) — granted the access roles they need, and granted to humans

The chain looks like:

```
User → ADVISOR (functional) → DATABASE_OBJECT_READ (access) → SELECT on [object]
```

When a new analyst joins, you grant them one functional role — not forty object privileges. When a new table needs exposure, you grant it to one access role — not to forty people.

Two pointers to remember

1. **Ownership** — *whoever creates an object owns it.* Create objects with the right role active (or transfer ownership afterward), or you end up with objects owned by personal roles that break when people leave.
2. **Future Grants** — `GRANT ... ON FUTURE TABLES IN SCHEMA ...`. **Without them, every newly created table silently lacks grants** and access tickets pile up. Future grants make new objects inherit the correct privileges automatically.

21. Masking, Row Access & Tags — Security *Inside* the Table

RBAC decides **who can touch the table** — but an enterprise needs control **inside the table**: which **columns** and which **rows**, automatically. Three instruments:

1. Dynamic Data Masking — column-level control

- A **policy attached to a column**.
- **Same query → different results by role.** The **data never changes; the view of it does.**
- Example: the **compliance team sees the account number as `12345678`, everyone else sees `********`** — from the same table, with no duplicate "masked copy" to maintain.

2. Row Access Policies — row-level security (RLS)

- A **predicate attached to the table's rows** — e.g., "regional managers see only their region's rows."
- **The filter lives in the table, not in every query someone remembers to write.** It cannot be forgotten, bypassed by a new dashboard, or omitted by a careless analyst.

3. Tag-Based Masking — classify once, protect forever

- **Classify once:** define a tag such as `PII_TYPE = 'SSN'`.
- **Attach the masking policy to the TAG** rather than to individual columns.
- Now **any future column tagged as PII is masked automatically the moment it's tagged** — you never explicitly manage masking column-by-column again. Governance scales with data growth instead of lagging behind it.

22. Security & Audit

The enterprise needs **forensic-grade access history and credential hygiene**. Three important features:

1. Access History

- The `ACCESS_HISTORY` view (in `ACCOUNT_USAGE`) provides **column-level forensics**: **who read which columns of which tables, when, via which query**. This is the view auditors and incident-response teams live in.

2. Key-Pair Authentication

- **Service accounts authenticate with a private key** — **no passwords** stored in config files, **no leaks** via shared credentials. (The same mechanism the Kafka connector used on Day 1.) Keys can be rotated without downtime.

3. Network Policies

- **IP allow-lists** enforced **per user or per account** — connections from outside approved networks are rejected outright.

Surrounding all of this: **SSO and MFA** for human users, and **encryption everywhere** — data is encrypted at rest and in transit, always, with hierarchical key management.

23. Snowpark — Send the Code to the Data

What it is

Snowpark is Snowflake's developer framework that lets you build **data pipelines and machine-learning workflows in Python, Java, and Scala directly inside Snowflake's engine**. It eliminates the need to extract, move, or transfer data to external servers — preserving both **processing speed and data governance**.

The inversion

- **The usual pattern:** bring the data *from* the database *to* the code (export to a Spark cluster, a VM, a laptop) — creating copies, latency, and governance gaps.
- **Snowpark inverts it: send the code to the data, where it lives.**

How it feels to use

- The **Python DataFrame API looks like Pandas**, but it **lazily compiles to SQL that runs on your Snowflake warehouse** — so **nothing ever leaves Snowflake**. Transformations are pushed down; only final results come back to the client.

The course's example

A volatility-calculation job becomes **a Python stored procedure running where the data lives** — **no export, no VM to maintain, full governance** (RBAC, masking, and access history all still apply, because the data never left).

24. Migration & Engineering Disciplines

Migration strategy

- **Prefer Snowflake's out-of-the-box migration services** for well-known source databases (**SQL Server, PostgreSQL, Oracle**).
- For **legacy warehouses and scripts**, a naive **"lift the SQL over" won't work** — legacy dialects carry quirks (proprietary functions, implicit conversions, procedural extensions) that need deliberate conversion.
- The course exercise: convert a deliberately quirky generic-legacy script to Snowflake SQL — and, critically, apply a **reconciliation process**: compare **row counts, sums, and checksums**

between the old-logic and new-logic outputs. Migration isn't done when the SQL parses; it's done when the numbers match.

Engineering habits that make a platform production-grade

- **SQL in Git version control** — every object definition, every transformation, reviewed and history-tracked.
- **Environments promoted Dev → Test → Prod** — and **Zero-Copy Clones make test environments essentially free** (clone prod, test against real-shaped data, drop).
- **Schema changes as reviewed migrations** — never ad-hoc ALTERs in prod.
- **Validation queries living in CI/CD pipelines** — correctness checks run automatically on every change.
- **DQ-Gate patterns** (data-quality gates) when pipelines are **orchestrated with Airflow** or similar tools — a pipeline stage that fails loudly when quality thresholds are breached, instead of silently propagating bad data downstream.

25. AI Evolution & the Snowflake AI Stack

The evolution timeline

Era	Paradigm	Characteristics
< 1990s	**Traditional AI**	Expert systems, rule-based systems — hand-coded logic
1990s–2000s	**Machine Learning**	Data-driven learning; applied mathematics & statistics; works with structured data; **manual feature engineering**
2005s–2020s	**Deep Learning**	Neural networks at scale, fueled by big data and GPUs; features learned automatically
2020s–2023	**Generative AI**	Large language models that create text, code, images
2024 →	**Agentic AI**	Adds **reasoning, multi-modality, automation, and tool calling** — AI that doesn't just answer, but plans and acts

The Snowflake AI Stack — bottom to top

1. Cortex AISQL Functions — pre-built LLMs as SQL functions

Call powerful language models the way you'd call `SUM()`:

- `CLASSIFY` — categorize text into labels you provide.
- `SENTIMENT` — score the sentiment of text.
- `COMPLETE` — general-purpose LLM completion against a prompt.
  All directly in SQL, on data that never leaves Snowflake's governance boundary.

2. Semantic Views + Cortex Analyst — grounded, not guessing

- A **semantic view** encodes your business definitions — what "revenue" means, how tables join, which filters apply.
- **Cortex Analyst** answers natural-language questions by generating SQL **grounded** in that semantic model — so the answers are **based on your actual definitions, not the model's guesses** about what your columns might mean.

3. Cortex Search — documents + RAG

- A managed search service over your **documents**, providing the retrieval layer for **RAG (Retrieval-Augmented Generation)** — letting LLM answers cite and draw on your enterprise content.

4. Cortex Agents — the orchestration platform

- Combines **Semantic Views + Cortex Analyst + Cortex Search + tool-calling abilities** into agents that can decide *which* capability a question needs — structured data, documents, or an external tool — and chain the steps to deliver the answer.

5. Cortex Code ("CoCo")

- An end-user agent surface for **SQL code explanation, fixing, and generation** — letting business users and developers perform SQL-related interactive operations conversationally.

Quick-Reference Cheat Sheet

Topic	The one-liner to remember
Architecture	Storage = Library, Warehouses = Reading Rooms, Cloud Services = Librarian
Pruning	"Partitions Scanned vs. Partitions Total"
Slow single query	Scale UP (bigger truck)
Queued queries	Scale OUT (more trucks)

Topic	The one-liner to remember
Suspension	Aggressive suspend for spiky loads; stay warm for steady trickle
Layers	Raw (as-arrived truth) → Staging (trusted) → Analytics (business-ready)
Reloadable landing tables	Make them TRANSIENT
Batch / Files / Rows	COPY INTO / Snowpipe / Snowpipe Streaming
Stage	A pointer to where files live
Storage Integration	The secure handshake (Snowflake ↔ IAM role)
VARIANT	Load now, define the shape later (schema-on-read)
Stream	A bookmark; DML clears the unread badge, SELECT doesn't
Task	Scheduled SQL where the data lives; `STREAM_HAS_DATA` makes it conditional
Dynamic Table	A SELECT plus a freshness promise (`TARGET_LAG`)
Streams+Tasks vs Dynamic Tables	Manual gear vs. automatic gear
Airflow	In-database steps → native; cross-system workflows → Airflow
Poor pruning	Fix with clustering, NOT a bigger warehouse
Spillage	The one case where a bigger warehouse helps
Join explosion	Fix the join logic — no hardware helps
Clustering / Search Opt / MV	Range scans / point lookups / repeated aggregations
Result cache	Zero-compute repeats — defeated by `CURRENT_TIMESTAMP`
Resource monitor	The credit circuit breaker
Time Travel	`AT` / `BEFORE` / `UNDROP` — fix disasters in seconds
Zero-Copy Clone	Metadata pointers, not data — pay only for divergence
Share	Live, read-only, revocable — no copy, no movement
RBAC	Access roles = WHAT; Functional roles = WHO; remember ownership & future grants

Topic	The one-liner to remember
Masking / RLS / Tags	Column view by role / filter lives in the table / classify once, mask forever
Snowpark	Send the code to the data
Migration	Reconcile row counts, sums, checksums — parsing isn't proving
Cortex stack	AISQL functions → Analyst (grounded) → Search (RAG) → Agents → CoCo